

<!-- Course: Hermes Architecture - Memory, Context & Gateways -->

*<!-- Source: VideoTranscriber_Hermes Architecture EXPLAINED_ Memory, Context & Gateways.txt -->
<!-- Pages: 7 | Generated per transcript-to-notes SKILL.md -->*

-- Page 1: Bird's Eye View - Hermes Agent Architecture

Header

🕒 ~4 min | Section 1 | 📖 Easy

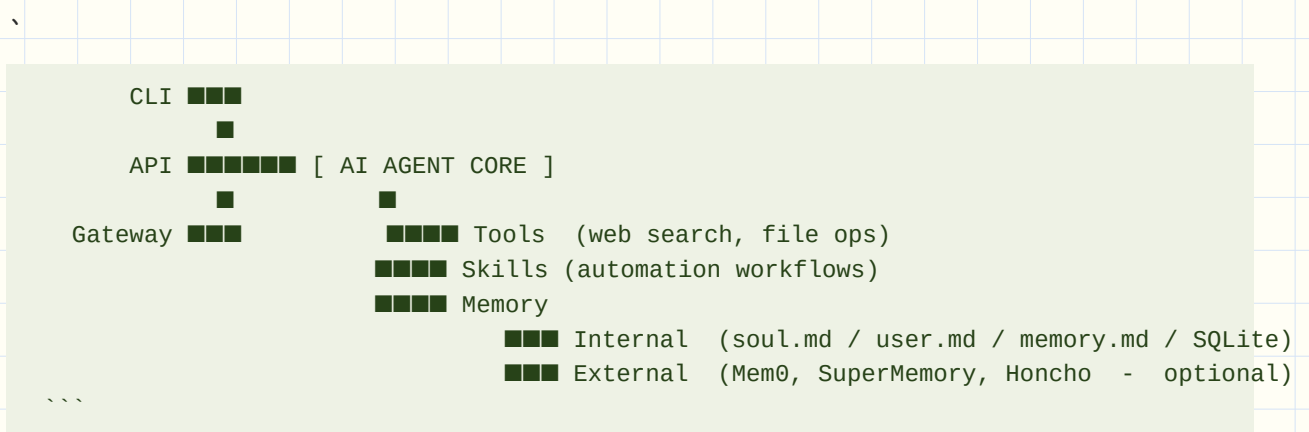
What You'll Learn

Understand every major component of the Hermes system at a glance so every subsequent section fits into a clear mental map.

Deep Dive Explanation

- Hermes is built around a single AI Agent Core (the agentic loop); all three entry points - CLI, Gateway, API - funnel into this same core without any separate logic per channel
- The CLI is synchronous and user-initiated: type hermes, get a response; the Gateway is always-on and event-driven, routing messages from Telegram, Slack, email etc.; the API is programmatic access - all three produce identical agent behaviour because the transport layer is decoupled from the reasoning layer
- Bundled services ship out-of-the-box: Tools (web search, file read/write), Skills (higher-level automation workflows), and a two-tier Memory system - no additional configuration required to use any of them
- Internal memory operates with zero configuration: it is purely local markdown files (soul.md, user.md, memory.md) plus a SQLite session transcript store
- External memory (Mem0, SuperMemory, Honcho) is an optional upgrade layer that adds long-term cross-session recall; without it, the agent still works fully - the difference is that it won't remember facts from sessions that have scrolled out of the context window

Visual Summary



Real-World Use First

Scenario: You install Hermes and immediately run hermes in your terminal. No gateway setup, no API key for memory providers - just the CLI. The agent loads its internal files (both empty) and responds using the default system prompt. After the conversation, user.md is updated with anything it learned about you.

Why it matters: Knowing the layered architecture lets you choose the right entry point and upgrade path - start with CLI, add a Telegram gateway when you want async access, add Mem0 when you want long-term recall.

Process Flow / Steps

1. User connects via CLI, Gateway, or API
2. Message reaches the AI Agent Core
3. Core assembles context from internal memory + prompts
4. Core calls the LLM, executes tools as needed
5. Returns response; updates internal memory

Key Concepts

- **AI Agent Core:** Central agentic loop - receives messages, builds context, calls the LLM, executes tools, and writes memory
- **Gateway:** Always-running service that bridges external messaging platforms to the agent core; each platform requires separate configuration
- **Internal Memory:** Local markdown files + SQLite transcripts; zero-config, always works offline
- **External Memory:** Optional third-party providers for long-term cross-session recall; must be explicitly configured at setup
- **soul.md:** Personality file; user-authored; empty by default (falls back to built-in Hermes identity)

Try This Right Now

- Open your Hermes installation and locate soul.md - note whether it is empty or populated
- Run hermes and ask "Who are you?" - if soul.md is empty you will see the default system identity

Flashcards

Q	A
---	---

Q	A
What are the three ways to connect to the Hermes agent?	CLI, Gateway, and API
What is the difference between internal and external memory?	Internal = local markdown files + SQLite (always available); External = optional providers like Mem0 for long-term recall
What service does soul.md control?	The agent's personality, tone, and goals
What happens if soul.md is empty?	The default built-in Hermes system prompt is used
Do external memory providers work by default?	No - they must be explicitly configured during setup

One-Page Revision

- Three entry points (CLI, Gateway, API) -> same AI Agent Core
- **Bundled services:** Tools, Skills, Memory - all out-of-box
- Internal memory = markdown files + SQLite (zero-config)
- External memory = optional long-term recall (Mem0, SuperMemory, Honcho)
- soul.md empty -> default identity; populated -> your custom agent persona



-- Page 2: The Agent Loop

Header

🕒 ~4 min | Section 2 | 📖 Easy

What You'll Learn

Understand the exact 5-step cycle Hermes follows every time you send a message - including the learning step at the end that makes it improve over time.

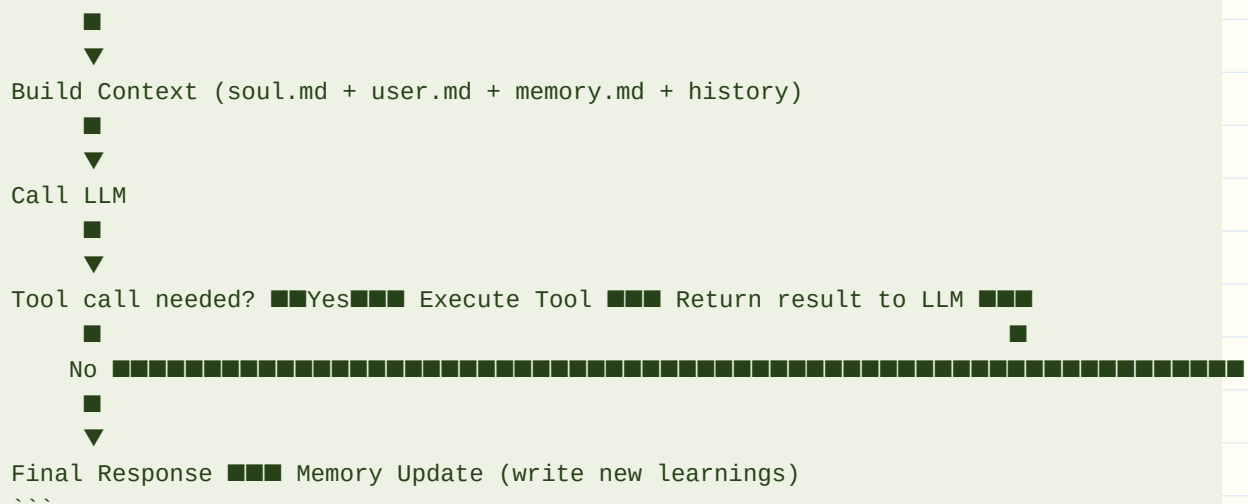
Deep Dive Explanation

- The loop runs on every single user message - there is no idle background reasoning; Hermes processes exactly when triggered
- **Step sequence:** Build Context -> Call LLM -> Execute Tools (sub-loop) -> Final Response -> Memory Update
- The tool execution phase is a sub-loop not a single step: the LLM outputs a tool call -> the tool executes -> the result is injected back into the LLM context -> the LLM decides whether to call another tool or produce a final answer; this repeats until no more tools are needed
- The memory update step runs after every response and is the mechanism that converts a stateless LLM call into a continuously learning agent - it analyzes the exchange and writes new learnings to user.md (user info) or memory.md (general facts)
- The design is intentionally minimalist - similar to PyAgent and OpenCode; simplicity makes the loop easy to reason about, debug, and extend

Visual Summary

,

User Message



Real-World Use First

Scenario: You ask Hermes "Research this week's AI papers and save the titles to a file." The LLM calls `web_search("AI papers this week")`, receives results, calls `write_file("ai_papers.txt", titles)`, then writes a summary response. After responding, the memory update notes your interest in AI research and saves it to `memory.md`.

Why it matters: The memory update step is what differentiates Hermes from a stateless chatbot - every conversation actively builds a richer model of you and your workflows.

Process Flow / Steps

1. User sends message
2. Hermes builds full context (markdown files + history)
3. Context sent to LLM
4. LLM calls tools as needed (sub-loop until done)
5. LLM produces final text response
6. Memory update: analyze exchange -> write learned facts to `user.md` or `memory.md`

Key Concepts

- **Agent Loop:** The deterministic 5-step cycle that runs for every user message; combines context assembly, LLM inference, tool execution, and memory writing
- **Tool Sub-loop:** The LLM may call multiple tools sequentially per turn; each result is fed back before the next decision
- **Memory Update:** Post-response step where Hermes identifies new user information or workflow insights and persists them to markdown memory files
- **Minimalist Design:** Loop is simple by intent - no parallel tool execution or built-in branching; matches the philosophy of PyAgent/OpenCode

Try This Right Now

- **Tell Hermes something personal:** "By the way, I'm a Python developer working on a data pipeline project"
- After the response, open user.md and verify that fact was captured

Flashcards

Q	A
---	---
When does the agent loop trigger?	Every time the user sends a message
What is the memory update step?	Post-response analysis that writes new user info or workflow insights into markdown memory files
Can the LLM call multiple tools per turn?	Yes - the tool sub-loop repeats until the LLM stops calling tools
What makes Hermes a "continuously learning" agent?	The memory update step that persists new learnings after every conversation
What other agents share a similar minimalist loop design?	PyAgent and OpenCode

One-Page Revision

- **Loop:** Build Context -> LLM -> Tools (sub-loop) -> Response -> Memory Update
- Memory update = the learning engine; runs after every response
- Tool calls repeat inside one turn - LLM decides when to stop
- **Minimalist design:** simple, debuggable, no parallel branches



-- Page 3: Context Building

Header

🕒 ~5 min | Section 3 | 📖 Medium

What You'll Learn

See exactly what Hermes packages into the context before every LLM call - why the order matters, and how user.md builds a personalized profile automatically.

Deep Dive Explanation

- Context is assembled in a fixed priority order that cannot be changed at runtime: soul.md first (personality foundation and highest-priority framing), then user.md (user profile), then memory.md (arbitrary learned facts), then optional external memory summaries, then skill/tool descriptions, finally the message history
- soul.md is the most impactful single file - it is the effective system prompt; leaving it empty means the LLM operates from a generic "Hermes virtual assistant" identity; a well-crafted soul.md can completely change tone, domain focus, and reasoning style
- **user.md is auto-maintained:** whenever Hermes identifies that the user revealed personal or professional information, it writes it to user.md without being asked; this profile accumulates over all sessions automatically
- memory.md stores workflow-level knowledge - patterns, procedures, preferences about tools and processes - not tied to the user personally, but to what Hermes has learned is useful in this environment
- Past conversation summaries from external memory only appear in context when an external provider is configured - without it, the context is lean and purely local
- Skill and tool descriptions are always included so the LLM always knows what capabilities are available

Visual Summary

,

Context window composition (every turn):

- 1. soul.md <- personality base ■
- 2. user.md <- auto user profile■
- 3. memory.md <- learned facts ■
- 4. Past summaries <- ext. memory only ■
- 5. Skill + tool descriptions ■
- 6. Message history <- or compressed ■

Real-World Use First

Scenario: After 3 months of use, your user.md contains: "works as a DevOps engineer," "prefers concise bullet responses," "uses Python and Bash." Every new question is answered with appropriate tool choices and concise formatting - no re-explanation needed.

Why it matters: The context design achieves persistent personalization without any active maintenance - it builds automatically as you use the agent.

Process Flow / Steps

1. Load soul.md as the system prompt foundation
2. Append user.md (auto-updated user profile)
3. Append memory.md (learned workflow facts)
4. If external memory configured: append relevant past conversation summaries
5. Append skill and tool descriptions
6. Append message history (or compressed summary if threshold reached)

Key Concepts

- **soul.md**: User-authored personality file defining tone, goals, and approach; empty = generic Hermes identity; this is the single most powerful customization lever
- **user.md**: Auto-populated user profile; Hermes writes it whenever it identifies user-specific information in conversation
- **memory.md**: Arbitrary fact and workflow store; Hermes writes here for knowledge that is useful in future conversations but not user-specific
- **Context order**: Priority is fixed - soul.md always frames everything before user content; tool descriptions always appear before message history

Try This Right Now

- **Write one line in soul.md:** "You are a senior DevOps coach who speaks concisely and uses real command examples"
- Ask Hermes "What should I learn next for my career?" - compare the response to before the change

Flashcards

Q	A
---	---
What is the first item added to the Hermes context?	soul.md (the personality / system prompt file)
Who updates user.md?	Hermes updates it automatically when it identifies user-specific information
What does memory.md store?	Arbitrary learned facts and workflow insights (not user-specific)
When do past conversation summaries appear in context?	Only when external memory is configured
What is soul.md by default?	Empty - falls back to built-in default Hermes identity

One-Page Revision

- **Context order (fixed):** soul.md -> user.md -> memory.md -> past summaries -> tools -> messages
- soul.md = highest-impact file; defines entire personality and framing
- user.md = auto-maintained; grows over sessions without manual input
- memory.md = workflow and fact learning; not user-specific
- External summaries only present when external memory provider is configured



-- Page 4: Context Compression

Header

🕒 ~6 min | Section 4 | 📖 Hard

What You'll Learn

Learn exactly how Hermes prevents context window overflow - covering two trigger points, two measurement methods, and what the compression prompt actually preserves.

Deep Dive Explanation

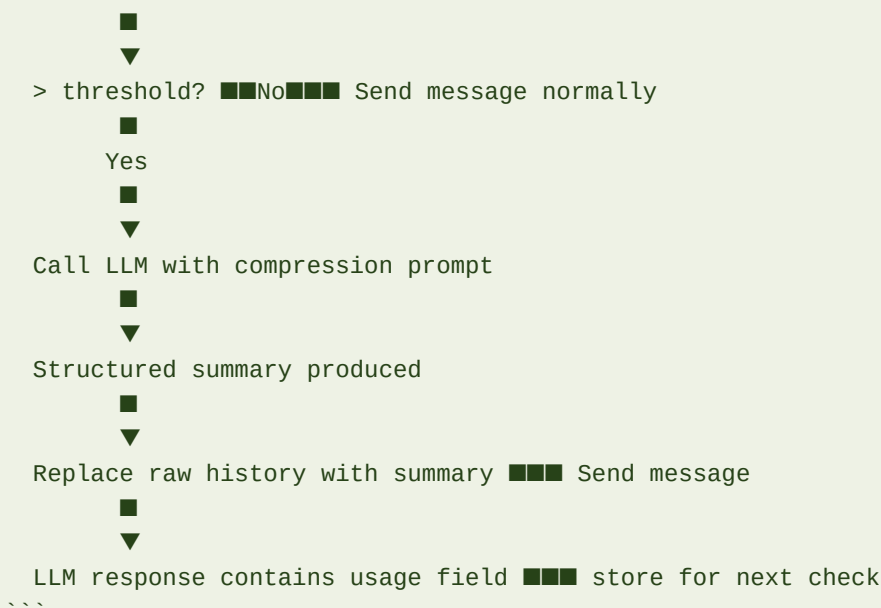
- Context compression triggers when accumulated token usage exceeds a configurable threshold (default 50% of the context window); this prevents overflow before it causes an API error
- Two trigger points provide both proactive and reactive protection: (1) before each turn - checked every time a message is about to be sent; (2) on LLM error response - acts as a safety net when the proactive check underestimates token usage
- Two measurement methods reflect the asymmetry between pre-send and post-response data: before the first message, token count is approximated as total characters $\div$ 4; after the first response, the LLM API usage field provides an accurate token count that is reused on subsequent turns
- Using the model's actual tokenizer at pre-send time would be accurate but computationally expensive - the char/4 heuristic is good enough (within ~20%) and costs nothing
- Hermes' compression prompt is structurally rich, far more detailed than Pi's minimalist summarizer - it preserves: overall goal, constraints, completed actions, active state, historical progress, blocked status, key decisions, resolved questions, relevant files, and critical context; this level of detail keeps the agent coherent after very long sessions
- After compression, the raw message history is replaced with the generated summary, dropping context usage dramatically (often back to ~10%)

Visual Summary

,

Before each turn:

Estimate tokens (char $\div$ 4 OR last LLM usage response)



Real-World Use First

Scenario: You're in a 2-hour architecture planning session with Hermes. After ~40 messages the context hits 50%. Before your next message, Hermes silently compresses: the LLM produces a structured summary capturing all design decisions, completed tasks, and open questions. You continue without interruption - Hermes still knows every decision made 90 minutes ago.

Why it matters: Without compression, long sessions either break with context overflow errors or require you to manually restart - compression is what makes Hermes viable for extended, professional-grade work sessions.

Process Flow / Steps

1. User sends message; before forwarding, estimate context size
2. First message: estimate = total chars ÷ 4; subsequent messages: use stored LLM usage token count
3. If estimate > threshold: call compression LLM with structured prompt
4. Receive structured summary (goals, decisions, actions, state, files, etc.)
5. Replace raw message history with summary in context
6. Proceed with sending message; save new usage count from LLM response
7. If LLM error (context overflow): re-check and force compression even if threshold not met

Key Concepts

- **Context compression:** Replaces accumulated message history with a structured LLM-generated summary when a token threshold is crossed
- **Char/4 heuristic:** Pre-first-message token estimate; cheap, no tokenizer dependency, accurate within ~20%

- **LLM usage response:** Post-response token count returned in the API response; accurate measurement used for all subsequent threshold checks
- **Compression threshold:** Configurable percentage (default 50%); raise to 70-80% for smaller models or cost-sensitive deployments
- **Structured compression prompt:** Produces a rich multi-section summary (not a flat recap) preserving goals, decisions, history, and active state

Try This Right Now

- Start a Hermes session with a small-context model; set threshold to 30% in settings
- Have a long, detailed back-and-forth; watch for the slightly longer pause that occurs when compression fires
- Send a question about an early message - verify Hermes still answers correctly from the summary

Flashcards

Q	A
---	---
What is the default context compression threshold?	50% of the context window
What are the two trigger points for compression?	Before each turn (proactive) and on LLM error response (reactive)
How is token count estimated before the first message?	Total characters divided by 4
What is the more accurate measurement used after the first message?	The LLM usage token count returned in the API response
What does the Hermes compression prompt preserve?	Goals, constraints, completed actions, active state, decisions, resolved questions, relevant files, and critical context

One-Page Revision

- **Default threshold:** 50% (configurable; 70-80% for smaller models)
- **Trigger 1:** before each turn (proactive); **Trigger 2:** on LLM error (reactive)
- **Pre-message token estimate:** $\text{char} \div 4$; post-message: use LLM response usage field
- Hermes compression = rich structured summary (not a flat recap)
- **After compression:** raw history replaced, context usage drops sharply



-- Page 5: The Gateway

Header

⏱ ~6 min | Section 5 | 📖 Hard

What You'll Learn

Understand how the always-running Gateway bridges external messaging platforms to Hermes - including how it reconstructs full conversation history from a single incoming message.

Deep Dive Explanation

- The Gateway is a persistent async process running independently of the agent loop - it simultaneously listens to all configured platforms without blocking the agent
- Each platform requires independent configuration because each uses a different protocol: Telegram uses short-polling (a loop every ~1 second calling the Telegram API), Discord can use webhooks or websockets, email uses IMAP/SMTP polling
- When a message arrives (e.g., from Telegram), the Gateway receives only the single new message - the platform does not send conversation history; the Gateway must reconstruct history from the local SQLite database using a composite session ID
- **Session ID composition:** platform::telegram_session_id::user_id - this namespacing means one SQLite database safely stores conversations from all platforms and users without any collision
- The session manager solves the concurrency problem: if a new message arrives while the agent is still processing a previous one, the session manager evaluates whether to interrupt (cancel current task, start fresh), steer (redirect the current task), or queue (hold the new message until the current one finishes)
- Users trigger these modes explicitly: /interrupt cancels; /steer redirects; a plain message is queued

Visual Summary

,

```

Telegram ■■■■ (short-poll every ~1s)
Discord ■■■■ (webhook or websocket)
  Slack ■■■■ (websocket)
  Email ■■■■ (IMAP poll)
WhatsApp■■■■
  ■
  ▼
  asyncio Gateway Loop
    ■
    Receive single message
      ■
      Build session ID key: platform::session::user
        ■
        Query SQLite -> load full history
          ■
          Build context (history + soul.md + user.md ...)
            ■
            Pass to AI Agent Core
              ■
              Session Manager: busy? -> interrupt / steer / queue
...

```

Real-World Use First

Scenario: On holiday, you send "Summarize my last project discussion" to your Hermes Telegram bot. The Gateway receives just that sentence + Telegram session ID. It looks up all prior Telegram messages in SQLite under telegram::<session_id>::<user_id>, reconstructs the full history, builds context, and passes it to the agent - which responds with accurate project context despite you being on a different device weeks later.

Why it matters: Without SQLite session reconstruction, every gateway message would be treated as a fresh conversation with no history.

Process Flow / Steps

1. asyncio loop starts; opens platform-specific listeners or pollers
2. Message arrives (e.g., Telegram sends one new message)
3. Gateway extracts platform name + session ID + user ID
4. Queries SQLite: `SELECT * FROM messages WHERE session_id = 'telegram::xxx::yyy'`
5. Loads full prior history; appends new message
6. Builds full context (history + markdown files + tool descriptions)
7. Passes to AI Agent Core; session manager handles concurrency

Key Concepts

- **Gateway:** Always-on asyncio service routing messages from external platforms into the AI Agent Core

- **Composite session ID:** platform::session::user key used to retrieve isolated conversation history from SQLite
- **Session Manager:** Concurrency handler that decides how to handle new messages when the agent is already busy (interrupt / steer / queue)
- **Home messaging platform:** The designated platform that receives cron job output and system-level notifications
- **Short-polling:** Telegram-style approach - tiny loop calls the platform API every ~1 second to check for new messages

Try This Right Now

- Configure Telegram as a gateway (hermes setup gateway)
- Send a message on Telegram, then in your terminal; verify both reach the same agent and share no history (different session IDs)
- Try /interrupt mid-response to observe the session manager cancellation

Flashcards

Q	A
---	---
What does the Telegram Gateway receive per incoming message?	Only the single new message - no prior history
How does the Gateway reconstruct conversation history?	By querying SQLite using the composite session ID (platform + session + user IDs)
What happens when you send a message while Hermes is already working?	The session manager decides: interrupt, steer, or queue
What command interrupts an in-progress Hermes response via Telegram?	/interrupt
Why does each platform need separate Gateway configuration?	Each uses a different protocol (polling, webhook, websocket, IMAP)

One-Page Revision

- Gateway = async loop listening to all configured platforms simultaneously
- **Each platform:** separate setup required (polling / webhook / websocket)
- Receives only the new message -> reconstruct history from SQLite via session ID
- Session ID = platform::session::user -> prevents history collision across platforms
- **Session manager:** /interrupt cancels, /steer redirects, plain message queues



-- Page 6: Memory Architecture

Header

🕒 ~5 min | Section 6 | 📖 Medium

What You'll Learn

See how Hermes organizes three distinct memory tiers, when each tier is queried, and the specific design choice behind deferring external memory queries to Turn 2.

Deep Dive Explanation

- Hermes memory is a three-tier system, each tier with different latency, capacity, query cost, and configuration requirements
- Tier 1 - Markdown files (soul.md, user.md, memory.md): loaded on every single turn via file read - zero query latency; limited only by what fits in the context window
- **Tier 2 - SQLite**: stores complete verbatim transcripts of every session (full record format) plus a bare-text table (raw message text only, no metadata) designed for fast similarity search; the bare-text table avoids the cost of deserializing full records just to search
- Tier 3 - External memory providers (Mem0, SuperMemory, Honcho): not active by default; each provider works differently (Mem0 uses similarity search, SuperMemory sends full history to an LLM for extraction, Honcho uses its own approach); all are queried after the first response of each conversation, not at Turn 1
- The "wait one turn" query strategy for external memory is deliberate: on Turn 1 the agent doesn't yet know what the conversation is about; waiting until after Turn 1 ensures the search query is semantically grounded - this mirrors how humans recall related memories after hearing a question, not before
- External memory results then appear in the context from Turn 2 onward, so follow-up questions benefit from surfaced memories

Visual Summary

Turn 1 Turn 2+

Tier 1	Markdown	✓ always	✓ always
Tier 2	SQLite	✓ (gateway)	✓ (gateway history)
Tier 3	External	✗ not yet	✓ queried after T1

External query fires after Turn 1 response,
results appear in context from Turn 2 onward.

Real-World Use First

****Scenario:**** 2 months ago you discussed AWS ECS deployment in detail; that session was stored

Why it matters: The deferred query prevents irrelevant or off-topic memories from polluting the context before the conversation topic is established.

Process Flow / Steps

1. Every turn: load Tier 1 markdown files (file read, zero latency)
2. Gateway messages: load Tier 2 SQLite history by composite session ID
3. Turn 1 runs: LLM responds without external memory
4. After Turn 1 response: query Tier 3 external memory using current context as the search signal
5. Tier 3 results appended to context - available from Turn 2 onward
6. End of every turn: Memory Update writes new learnings back to user.md or memory.md

Key Concepts

- **Tier 1 - Markdown:** Always in context; zero query cost; limited to context window size; soul.md + user.md + memory.md
- **Tier 2 - SQLite:** Full session transcripts + bare-text similarity table; used by Gateway for history reconstruction
- **Tier 3 - External memory:** Optional providers; queried post-Turn-1 to ensure topic context before searching
- **Bare-text table:** Dedicated SQLite table with only raw message text - optimized for fast similarity search without loading full record objects
- **Deferred query strategy:** External memory searched after Turn 1, not before - ensures search has semantic grounding from established conversation topic

Try This Right Now

- Enable a free-tier external memory provider (e.g., Mem0 free tier) in Hermes setup
- Have a conversation about a specific topic you've discussed before
- On Turn 2, observe if Hermes references past context from that earlier session

Flashcards

Q	A
---	---

Q	A
What are the three memory tiers in Hermes?	Tier 1: Markdown files; Tier 2: SQLite session store; Tier 3: External memory providers
When is external memory first queried in a conversation?	After the first response (available from Turn 2 onward)
Why is external memory not queried on Turn 1?	The topic isn't established yet - the deferred query ensures semantically grounded search
Why does SQLite have a bare-text table?	To enable fast similarity search without deserializing full session records
Is Tier 3 active by default?	No - must be explicitly configured during setup

One-Page Revision

- **Tier 1:** markdown (always in context, zero latency)
- **Tier 2:** SQLite - full transcripts + bare-text similarity table (gateway history)
- **Tier 3:** external providers - optional; queried after Turn 1, results from Turn 2
- Deferred query = semantically grounded search; avoids polluting context prematurely
- **Providers:** Mem0 (similarity), SuperMemory (LLM extraction), Honcho - each unique



-- Page 7: Cron Jobs

Header

⏱ ~4 min | Section 7 | 🟢 Easy

What You'll Learn

Understand how Hermes automates recurring tasks with an internal tick loop - including a key discrepancy between the docs and actual implementation.

Deep Dive Explanation

- Hermes cron is a self-managed internal loop, not OS-level cron - a function called `tick()` runs every 60 seconds as part of the Hermes process; no system scheduling permissions required
- Despite official documentation stating jobs are stored in SQLite, the actual implementation reads from `~/.hermes/cron/jobs.json` - a plain JSON file; this discrepancy was found by reading the source code directly
- Each job entry contains a schedule expression (cron syntax), a prompt (the instruction Hermes will execute), and metadata; when `tick()` matches the current minute to a job's schedule, it runs that prompt as a full agent loop call
- Execution results are saved as `run.md` inside `~/.hermes/cron/output/<job-id>/` - providing a complete auditable history of every scheduled run per job
- Cron result delivery bypasses the `send_message` tool entirely - results go directly to the designated "home" messaging platform via the Gateway's system channel; this means delivery is reliable even if the agent loop would otherwise be blocked

Visual Summary

,

Every 60 seconds:

`tick()`

```

■
▼
Read ~/.hermes/cron/jobs.json
■
▼
Any job matches current minute? ■■No■■■ Wait for next tick
■
Yes
■
▼
Run job as full agent loop (build context + LLM + tools)
■
▼
Save result: ~/.hermes/cron/output/<job-id>/run.md
■
▼
Send to home messaging platform (NOT via send_message tool)
...

```

Real-World Use First

Scenario: You configure: "Every weekday at 7:00am, find the top 5 AI news headlines and format a morning briefing." At 7:00am Monday, tick() fires, reads jobs.json, runs the web search + formatting as a full agent loop, saves the result, and pushes it to your Telegram home channel - you wake up to a formatted briefing with no action needed.

Why it matters: Cron transforms Hermes from a reactive assistant (answers when asked) to a proactive one (acts on schedule autonomously).

Process Flow / Steps

1. tick() fires every 60 seconds
2. Reads ~/.hermes/cron/jobs.json
3. Compares each job's schedule expression to current time
4. On schedule match: executes job prompt as full agent loop (context + LLM + tools)
5. Saves result to ~/.hermes/cron/output/<job-id>/run.md
6. Delivers output to configured home messaging platform via Gateway system channel

Key Concepts

- **tick():** Internal 60-second loop function; the heartbeat of Hermes cron; independent of OS cron daemon
- **jobs.json:** Actual job storage file - plain JSON at ~/.hermes/cron/jobs.json (not SQLite, despite documentation)
- **Home messaging platform:** The gateway integration designated to receive cron output and system-level notifications

- **run.md**: Per-execution result file saved inside the job's output directory - auditable history of every run
- **Direct gateway delivery**: Cron bypasses send_message tool; output goes directly through the Gateway system channel

Try This Right Now

- Open ~/.hermes/cron/jobs.json - inspect the JSON structure; note the schedule and prompt fields
- **Create a test job**: schedule * * * * *, prompt "Say the current time" - verify run.md appears in the output directory after ~60 seconds

Flashcards

Q	A
---	---
How often does Hermes cron tick() fire?	Every 60 seconds
Where are cron jobs actually stored?	~/.hermes/cron/jobs.json (plain JSON - not SQLite as the docs say)
How does Hermes deliver cron results to the user?	Directly to the configured home messaging platform via the Gateway system channel
Where are cron execution results saved?	~/.hermes/cron/output/<job-id>/run.md
Can a cron job apply conditional logic (e.g., only alert on failure)?	Yes - each job runs as a full agent loop; the LLM can decide whether to produce output

One-Page Revision

- Cron = internal tick() loop every 60 seconds (not OS cron)
- Jobs stored in jobs.json - docs say SQLite but code says JSON
- **Each job**: schedule expression + prompt -> full agent loop execution
- Results saved to run.md; delivered to home platform (NOT send_message tool)
- Enables proactive, schedule-driven autonomous behaviour